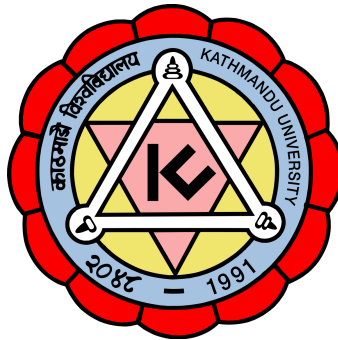


Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



A PROJECT REPORT ON
**REAL-TIME VOLUMETRIC CLOUD RENDERING
USING RAYMARCHING IN ODIN**

Subject: Computer Graphics (COMP342)

Submitted by:

Sadit Rasaili (Roll No. 40)

Samyak Maharjan (Roll No. 23)

Submitted to:

Mr. Dhiraj Shrestha

Department of Computer Science and Engineering

Submission Date: 2026/08/04

1. Introduction

1.1 Background

Clouds are among the most recognisable examples of a volumetric phenomenon: soft, translucent, and constantly changing, they cannot be represented convincingly with ordinary opaque, polygon-based surfaces. Real-time rendering of clouds is therefore usually approached with raymarching, a technique in which a ray is stepped through a three-dimensional density field and the light reaching the camera is accumulated sample by sample, rather than rendering a single solid mesh. This project implements such a raymarched volumetric cloud renderer from first principles, without the aid of a game engine's built-in volumetric or shader-graph tooling.

The renderer was built using the Odin programming language together with the raylib multimedia library for windowing and graphics, and microui for an in-application, real-time parameter panel.

1.2 Motivation

Game engines such as Godot provide ready-made spatial shader pipelines, material systems, and scene graphs that handle much of the bookkeeping around a custom shader. Building the same renderer directly on top of raylib and a GLSL shader pair removes that layer: the shader program, its uniform locations, the render textures, the resize handling, and even the on-screen parameter controls all have to be written and wired up by hand. Doing so gives a much closer view of how a raymarched volumetric effect actually talks to the GPU, and was the main motivation for building a second, independent implementation of the same technique.

1.3 Objectives

- Implement a real-time raymarched volumetric cloud renderer capable of rendering multiple procedurally generated cloud clusters composed of smoothly blended signed distance field (SDF) primitives.
- Model absorption, self-shadowing, anisotropic scattering, and edge “powder” darkening so the cloud responds believably to a movable sun.
- Expose the renderer's parameters through an interactive, in-application control panel for live experimentation rather than recompiling to see a change.
- Support hot-reloading of the shader source so shader iteration does not require restarting the application.
- Keep the renderer responsive to window resizing without breaking the layout of the scene view and the control panel.

1.4 Scope

The project covers a multi-cloud, single-light raymarched renderer in which each cloud is generated procedurally from multiple smoothly blended SDF primitives. It does not attempt terrain integration, cloud shadows cast onto a ground plane, dynamic weather systems, or physically based volumetric weather simulation.

2. Project Description

2.1 Technology Stack

Odin is a systems programming language designed as a modern alternative to C, with manual memory management, built-in array/slice types, and strong support for calling into C libraries; it was used here for all application logic. raylib is a small, C-based library for windowing, input, texture loading, and shader management, exposed to the project through Odin's vendor:raylib bindings and used for every graphics call in the program. microui is a very small immediate-mode GUI library (bound through vendor:microui) that was used to build the on-screen “Cloud Parameters” panel; being immediate-mode, its widgets are re-declared every frame directly from the current parameter values, with no separate retained widget tree to keep in sync.

2.2 System Architecture

The application is a single Odin source file organised around a small number of data structures. A Cloud_Params struct holds every tunable rendering value sphere radius, absorption, anisotropy, march size, density scale, step counts, sun direction and colour, sky colours, and ten boolean feature toggles together with a default_params() constructor. A Shader_State struct wraps the compiled GLSL shader program and the two supporting textures (a tileable noise texture and a blue-noise texture) along with their uniform locations. A thin glue layer adapts raylib's mouse, keyboard, and clipboard events into microui's input model and renders microui's draw commands directly into the active framebuffer. The main loop ties these pieces together once per frame.

2.3 Rendering Pipeline: Raymarching

The fragment shader evaluates a procedural cloud field composed of multiple cloud clusters distributed throughout the scene. Each cluster is constructed from several overlapping signed distance field (SDF) spheres that are smoothly blended together to produce larger cloud formations. During raymarching, the density contributed by nearby clusters is accumulated while empty space is efficiently skipped using cluster bounding volumes.

2.4 Cloud Density and Noise

Cloud density is generated procedurally from two components. First, the overall cloud shape is formed by multiple overlapping signed distance field (SDF) spheres grouped into cloud clusters using smooth union operations. This creates large, irregular cloud volumes instead of a single spherical object. Secondly, a pre-generated tileable noise texture is sampled using fractal Brownian motion (FBM) to erode the cloud surfaces and introduce fine-scale detail. The noise field is animated over time when animation is enabled, causing the cloud formations to drift naturally. A blue-noise texture is additionally used to jitter raymarch sample positions, reducing visible banding artifacts.

2.5 Procedural Cloud Generation

Instead of representing a cloud as a single volumetric sphere, the renderer generates each cloud as a collection of overlapping SDF spheres positioned around a cluster center. Each sphere has a randomly generated radius and offset, producing irregular cloud silhouettes. Smooth union operations blend neighboring spheres into a continuous volume, eliminating visible intersections. Finally, FBM noise modulates the resulting density field to create the soft, turbulent boundaries characteristic of real cumulus clouds. Multiple cloud clusters are randomly distributed throughout the scene and animated using a common wind vector, allowing an entire cloud field to be rendered.

2.6 Lighting Model

Light transport through the cloud field is approximated with a small set of physically-inspired terms, each independently toggleable so its individual contribution can be inspected. The absorption parameter attenuates light exponentially as it passes through denser regions of the cloud field, in the spirit of Beer's law. A secondary “light march” of up to `max_light_steps` additional samples is cast from each primary sample point toward the sun to estimate self-shadowing, so that thicker parts of the cloud field darken the regions behind them relative to the sun. The anisotropy parameter weights forward versus backward scattering of light, approximating a Henyey–Greenstein-style phase function and producing the bright “silver lining” effect seen when looking toward the sun through cloud field edges. A `powder_strength` term darkens cloud field edges nearest the camera, reproducing the visual “powder” or “sugar” effect seen in dense, backlit cloud field silhouettes; an ambient term adds a flat fill light so that fully self-shadowed regions are not rendered pure black; and `sun_glow_exponent` controls how tight or broad the glow halo around the sun disc appears.

2.7 Sky and Atmosphere

Behind the cloud volume, a simple procedural sky is drawn as a vertical gradient interpolated between a `sky_top` and a `sky_bottom` colour, both independently adjustable. An optional tonemap toggle applies a final tone-mapping pass to the composited image, and a `cloud_tint` colour can be used to bias the overall colour of the cloud independently of the lighting terms above.

2.8 Interactive Control Panel

All of the parameters described above are exposed live through a microui panel docked to the right edge of the window. The panel's width is computed every frame as a percentage of the current window width, clamped between a minimum and maximum, so it scales sensibly as the window is resized. It is organised into collapsible sections which includes:

- Ten independent feature toggles: Clouds, Animate, Self-Shadow, Powder Edges, Anisotropy, Sun Glow, Sky Gradient, Dither, Tonemap, and a “Fast Preview” low-quality-noise mode, so any single technique can be isolated for comparison.
- Sliders for radius, absorption, anisotropy, march size, density, noise speed, step counts, ambient, powder strength, sun glow exponent, noise scale, and cloud tint.
- Grouped RGB sliders for sun colour and the top/bottom sky colours, and XYZ sliders for sun direction.
- A “Reset to Defaults” button that restores every value in one step.

Every change made in the panel writes directly into the shared `Cloud_Params` struct, which is then re-uploaded to the shader's roughly thirty uniform locations on the very next frame there is no separate “apply” step, so the rendered cloud updates as the slider is dragged.

2.9 Two-Pass Frame Loop

Each frame is drawn in two passes. The first renders the cloud scene into an off-screen render texture sized to only the “scene” region of the window the full window width minus the docked panel's width with the cloud shader bound and the noise textures attached. The second pass composites that texture onto the visible framebuffer (flipped vertically, since raylib's render textures are stored bottom-up), draws a thin divider line, renders the microui panel on top, and overlays an FPS counter and a short reload hint. The off-screen render target is recreated automatically whenever the scene region's size changes, keeping the cloud render resolution matched to the available viewport rather than the full window.

2.10 Development Workflow

Pressing F5 reloads and recompiles `shaders/vertexShader.glsl` and `shaders/fragmentShader.glsl` directly from disk and swaps in the new shader program without restarting the application or resetting the current parameter values, considerably shortening the edit-compile-look loop while tuning the shader. F11 toggles fullscreen. Window size is re-read from the real framebuffer every frame rather than relying on raylib's `resize-event` flag, which the project notes can miss the frame in which a maximize or fullscreen transition happens; reading the live render size also correctly accounts for high-DPI scaling.

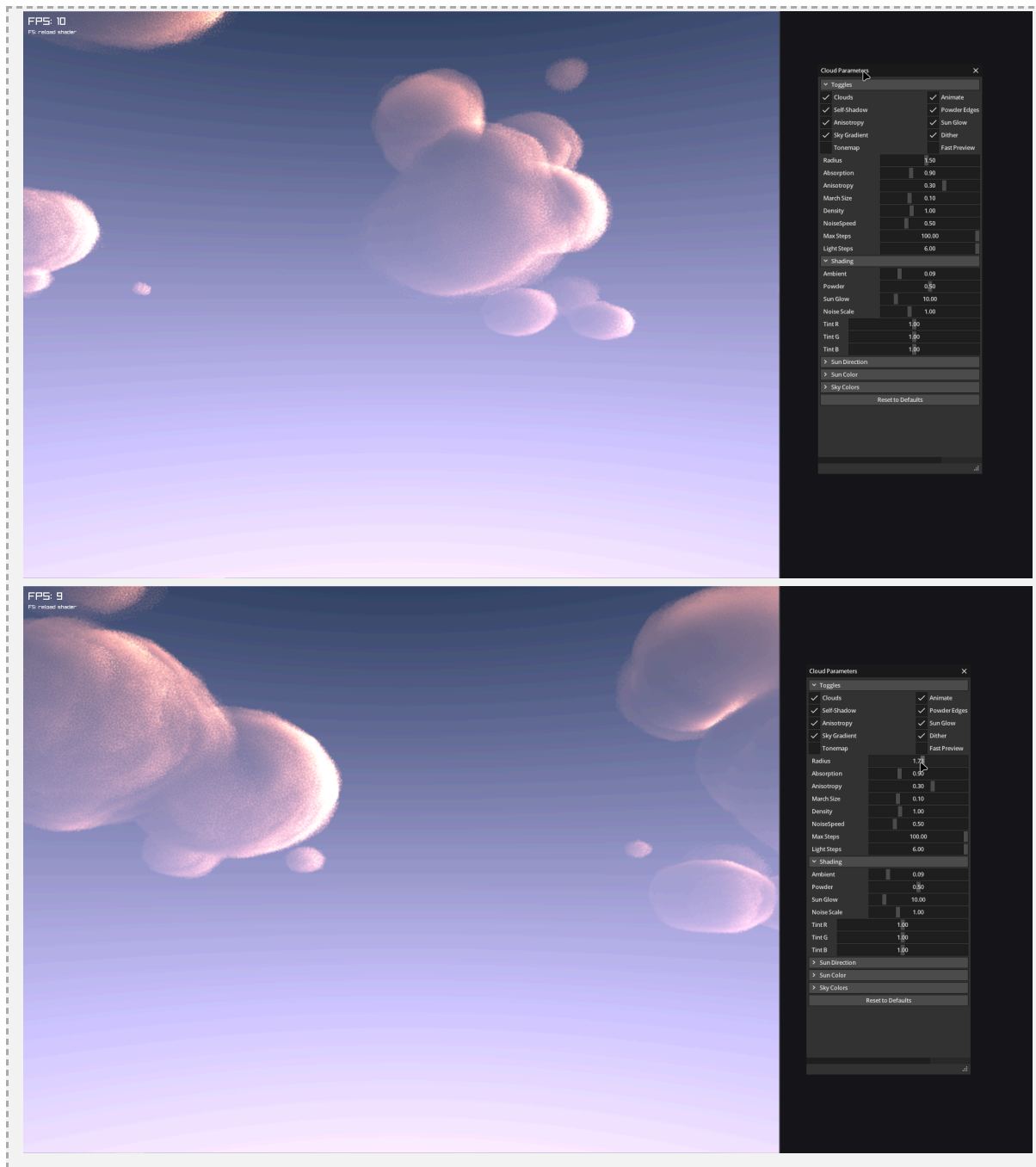


Figure 1: Application window showing the rendered cloud volume alongside the Cloud Parameters panel.

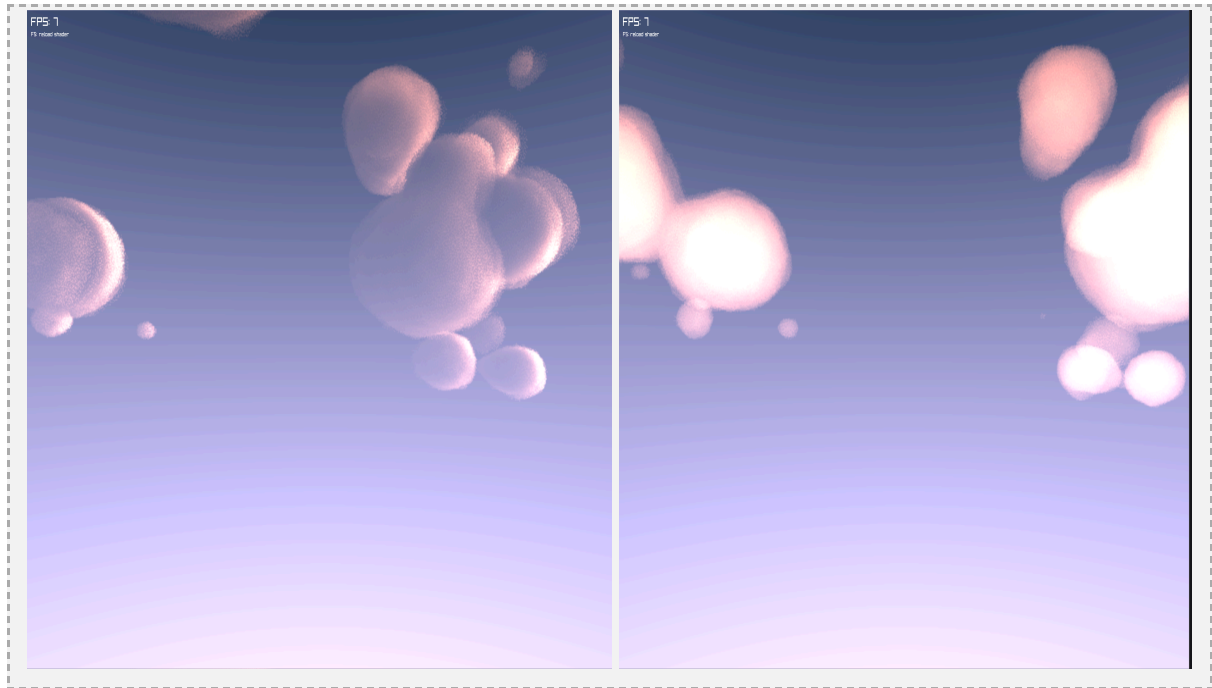


Figure 2: Effect of toggling self-shadowing (light march) and powder edges on and off.

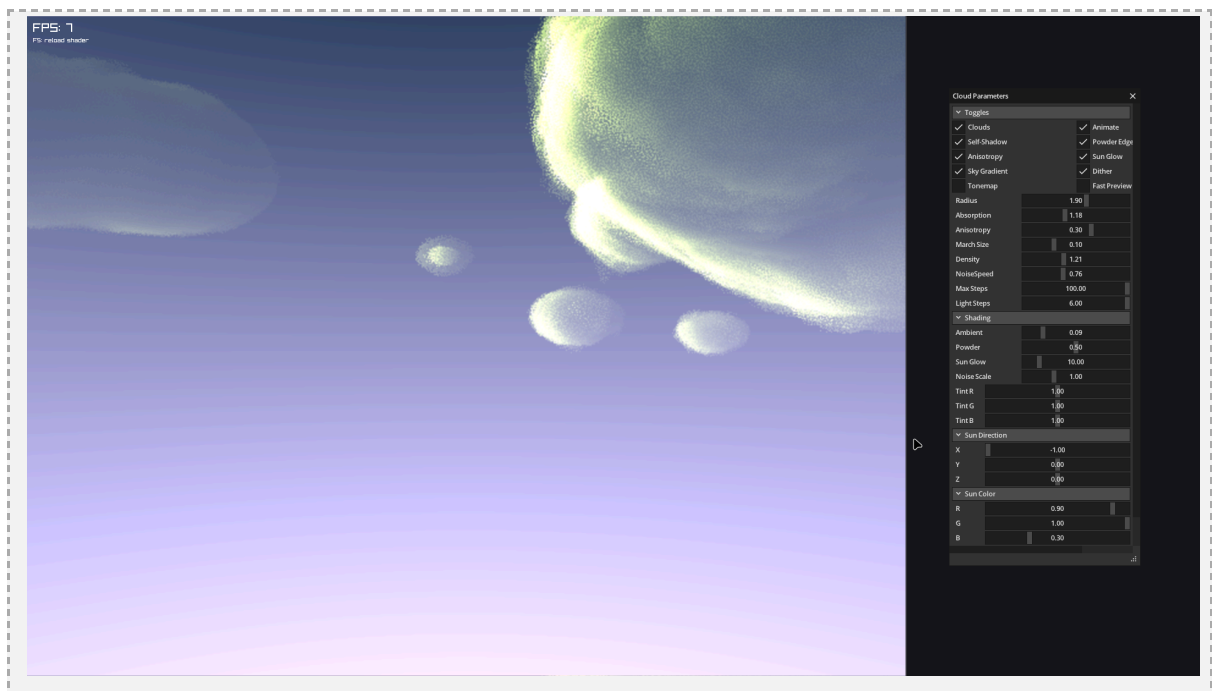


Figure 3: Sky gradient, sun colour, and sun direction being adjusted live through the panel.

3. Conclusion

This project produced a self-contained, real-time volumetric cloud renderer capable of rendering a procedural field of cloud formations instead of a single volumetric object. By combining raymarching with multiple smoothly blended signed distance field primitives and fractal noise modulation, the renderer generates more natural cloud silhouettes while preserving interactive control over the rendering parameters.

Working through the lighting model in particular reinforced why each term is normally present in production cloud shaders: self-shadowing via a secondary light march is what keeps thick clouds from looking flat, blue-noise dithering is what keeps a comparatively low step count from banding visibly, and the anisotropic scattering term is what produces the bright edge seen when the sun sits behind a cloud.

A number of extensions were left out of scope for this iteration and are natural next steps: replacing the 2D noise texture with a baked or compute-shader-generated 3D Worley/FBM volume for less repetitive cloud shapes, adding temporal reprojection or upsampling to allow higher step counts at interactive frame rates and casting the cloud's shadow onto a ground plane or terrain.

4. References

- [1] Rasaili, S. and Maharjan, S. (2026). volumetric_clouds_odin [Source code]. GitHub.
https://github.com/Rasaili-rain/volumetric_clouds_odin
- [2] Odin Programming Language - official site and documentation. <https://odin-lang.org/>
- [3] raylib - a simple and easy-to-use library to enjoy videogames programming.
<https://www.raylib.com/>
- [4] microui -a tiny, portable immediate-mode UI library (rxi). <https://github.com/rxi/microui>
- [5] Lague, S. Coding Adventure: Clouds. YouTube.
<https://www.youtube.com/watch?v=4QOcCGI6xOU>
- [6] Heckel, M. Real-time dreamy Cloudscapes with Volumetric Raymarching.
<https://blog.maximeheckel.com/posts/real-time-cloudscapes-with-volumetric-raymarching/>
- [7] Hillaire, S. (2016). Physically Based Sky, Atmosphere and Cloud Rendering in Frostbite. SIGGRAPH 2016 Physically Based Shading Course.
- [8] Henyey, L. G. and Greenstein, J. L. (1941). Diffuse Radiation in the Galaxy. Astrophysical Journal, 93, 70–83.